Interval Verification Documentation for `TrapeziumFence`

July 2026

Abstract

This document describes the architecture, verification logic, functions, and recorded experiments for the `TrapeziumFence` package. The package verifies upper bounds for the normalized shortest equal-area fence in convex quadrilaterals and also records necessary-condition exclusions for optimality. The search subdivides a four-dimensional rectangle of coordinates for the two free vertices C, D. Each leaf rectangle is either discarded as certainly inadmissible, certified as having a shortest fence value below a threshold θ, certified nonoptimal by the opposite-pair equality condition, or retained as an unresolved survivor. The certificate verifier re-checks both the metadata, local leaf claims, and global dyadic tiling of the original domain.

Contents

1 Mathematical Normalization

The package studies convex quadrilaterals $DCBA$ with $D = (0, 0)$, $C = (1, 0)$, $B = (b_1, b_2)$, and $A = (a_1, a_2)$. The side DC is assumed to be a longest side and is normalized to length 1. This loses no generality for the target functional, because fence lengths are divided by $\sqrt{\text{Area}(DCBA)}$. A quadrilateral with a longer side can be rescaled and appears in another copy of the normalized search.

The raw search box is

$$b_1 \in [0, 2], \qquad b_2 \in [0, 1], \qquad a_1 \in [-1, 1], \qquad a_2 \in [0, 1].$$

The admissibility predicate keeps only boxes that may contain counterclockwise convex quadrilaterals whose non-base side lengths are at most 1:

$$|CB| \le 1, \qquad |BA| \le 1, \qquad |AD| \le 1.$$

The C implementation and JSONL records still use the legacy coordinate slots (c_1, c_2, d_1, d_2). In the public notation of this document these slots mean (b_1, b_2, a_1, a_2).

For a quadrilateral $Q = DCBA$, the code computes six scale-invariant fence-construction candidate values $F_i(Q)$. Each candidate value is an upper bound for the normalized shortest fence value, so

$$f(Q) \le \min_{0 \le i < 6} F_i(Q).$$

The six items are four vertex-angle candidates and two exact opposite-edge-pair candidates. A box is certified below θ when interval arithmetic proves that the upper endpoint of an enclosure for $\min_i F_i$ is at most θ. Consequently, certified boxes are one-sided exclusions for the problem $f(Q) > \theta$: they prove that no admissible quadrilateral in the box has normalized shortest fence value above the threshold.

2 Package Architecture

The C implementation is arranged in layers:

`geom.c`, `geom.h` Arb arithmetic wrappers, four-variable forward-mode automatic differentiation jets, and basic point/vector operations.

`series.c`, `series.h` Rigorous enclosures for $g(\gamma) = \gamma / \sin\gamma$ and $g'(\gamma)$, used by the parallel-safe opposite-pair formula near $\gamma = 0$.

`functionals.c`, `functionals.h` Construction of the six explicit fence-construction candidates, natural and centered interval enclosures, the minimum envelope used by certification, and the optional `functionals\_opposite\_pairs\_disjoint` nonoptimality certificate. Items 4 and 5 are exact opposite-pair construction values, not arbitrary majorants; the nonoptimality certificate relies on this contract.

`admissible.c`, `admissible.h` Conservative interval tests that discard boxes containing no admissible normalized quadrilateral, plus the flat/small-area certificate `box\_small\_area\_certifies\_lo`

`threshold.c`, `threshold.h` Exact decimal threshold parser and rational comparison helpers. The literal passed to `--theta` is the mathematical constant used by certification; printed doubles are only diagnostic.

`search.c`, `search.h` Dyadic subdivision engine. It classifies boxes as discarded, certified low, flat-area low, nonoptimal, or surviving; splits unresolved boxes; collects statistics; and can seed a refinement run from previous survivors.

`cert.c`, `cert.h` JSONL certificate writer, stateless certificate verifier, and certificate assembler. The verifier first checks the self-describing metadata line, then recomputes local claims and checks that all leaves tile the original domain by the same dyadic splitting rule. Missing dyadic children fail immediately. The assembler replaces survivor leaves through a refinement chain and checks metadata compatibility across the chain.

`fence_validate.c` Command-line front end for searching, refining, verifying, and evaluating a point.

`analyze_cert.py` Safe postprocessing of certificate files. It does not replace certificate verification.

`analyze_refinement_distances.py` Distance-to-T^* postprocessor for the recorded refinement chain. It reports diagonal-style bounds from each survivor rectangle to the reference trapezoid.

`tests/test_trap.c` Unit tests for the reference trapezoid, pair-bound formula agreement, series evaluation, inadmissibility, one interior certification, the pair-equality certificate, and the flat-area certificate.

`tests/test_cert_cli.sh` CLI regression test for certificate verification, including fast failure on a missing certificate leaf.

3 Rectangle Validation Logic

3.1 Cells

A search cell is a four-dimensional rectangle

$$X = [b_1^-, b_1^+] \times [b_2^-, b_2^+] \times [a_1^-, a_1^+] \times [a_2^-, a_2^+].$$

Equivalently, it is a product of two planar rectangles:

$$X_A = [a_1^-, a_1^+] \times [a_2^-, a_2^+], \qquad X_B = [b_1^-, b_1^+] \times [b_2^-, b_2^+].$$

The initial cell has

$$X_A = [-1, 1] \times [0, 1], \qquad X_B = [0, 2] \times [0, 1].$$

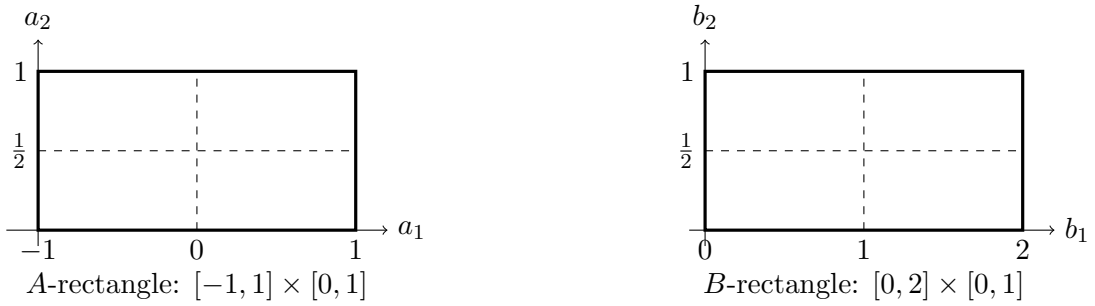

Figure 1: Equal-axis initial planar grids for A and B. Combining one rectangle from the A-grid with one rectangle from the B-grid gives a four-dimensional cell.

3.2 Splitting Rule

The root spans are

$$(2, 1, 2, 1)$$

for the public coordinates (b_1, b_2, a_1, a_2). The search compares scaled widths

$$\frac{b_1^+ - b_1^-}{2}, \qquad b_2^+ - b_2^-, \qquad \frac{a_1^+ - a_1^-}{2}, \qquad a_2^+ - a_2^-.$$

The coordinate with largest scaled width is bisected. If the coordinate is b_1 or b_2, only the B-rectangle changes; if it is a_1 or a_2, only the A-rectangle changes. This rule is deterministic and is used by both the search and the certificate verifier.

3.3 Leaf Classification

For a box X, the search applies the following logic:

1. If `box\_certainly\_inadmissible` proves that the entire box violates convexity, side-length, or optional half-symmetry constraints, the leaf is written with status `discarded`.

2. If `--flat-area-cert` is enabled and `box\_small\_area\_certifies\_low` proves the flat/small-area estimate below, the leaf is written with status `flat\_area`. This is a genuine low-fence certificate.

3. If `--pair-eq-cert` is enabled and `functionals\_opposite\_pairs\_disjoint` proves that the two opposite-pair fence intervals are disjoint, the leaf is written with status `nonoptimal`. This excludes an optimizer by a necessary optimality condition, but is not by itself a low-fence claim.

4. Otherwise, `functionals\_min` computes an interval enclosure $[f_{\mathrm{lo}}, f_{\mathrm{hi}}]$ for the minimum of the six candidate values. If the enclosure is finite and $f_{\mathrm{hi}} \le \theta$, the leaf is written with status `certified`.

5. If the box is not certified and its largest scaled width is at most `wfloor`, it is written with status `survivor`. A survivor is not evidence of optimality; it is merely an unresolved part of the outer neighborhood.

6. Otherwise the box is split into two children and the process continues.

The search can be restarted from survivors by passing `--refine FILE`. Refinement outputs are partial certificates. Use `--assemble` to replace survivor leaves stage by stage and produce a complete certificate for the original root box.

3.4 Self-Describing Certificate Metadata

Every JSONL certificate begins with a metadata line of schema version 3. It records the threshold θ as an exact decimal string, enclosure form, half-domain flag, auxiliary-certificate flags, precision provenance, root box, and splitter spans. Leaf lines follow this first line.

The verifier reads θ, `form`, and `half` from the file. Supplying `--theta`, `--form`, or `--half` during `--verify` is only a cross-check: if a command-line value disagrees with the certificate metadata, verification fails with `FAIL[meta]`. This prevents a certificate for one threshold from being silently audited as a different, usually weaker, statement.

The exact θ literal is parsed as a rational number. Certified leaves compare the Arb upper endpoint to this rational. The flat-area certificate compares the area upper endpoint to the exact rational $\theta^2/4$, not to a rounded floating-point value. The metadata field `theta\_approx` and printed decimal summaries are diagnostic only.

The assembler requires metadata in every input file and checks that the global claim parameters are compatible before it replaces survivor leaves. Old JSONL files without schema-3 metadata must be regenerated with the current program before they can be audited or assembled.

3.5 Certificate Semantics

A complete verified certificate proves:

every admissible quadrilateral outside the survivor union is either

certified below θ or certified nonoptimal.

Statuses `certified` and `flat\_area` are low-fence certificates: they prove $f(Q) \le \theta$ throughout the box. Status `nonoptimal` means that no optimizer lies in the box, because a necessary optimality condition fails. It does not assert a pointwise low-fence bound.

Therefore a certificate with `--pair-eq-cert` enabled should be read as an exclusion certificate for possible above-threshold optimizers, not simply as a partition into low boxes and survivors. It does not prove that survivor boxes contain values above θ, nor does it produce a lower bound for the true optimum. It gives an outer neighborhood for currently unresolved candidate configurations.

4 Auxiliary Certificates

4.1 Opposite-Pair Equality

The two opposite-pair candidate values are exact fence-construction values for the pairs (DC, BA) and (CB, AD). A theoretical necessary condition for optimality is that these two fences agree at an optimizer. The optional certificate `--pair-eq-cert` evaluates interval enclosures for the two exact opposite-pair candidate values. If the finite intervals are disjoint, then no point in the box can satisfy the equality condition, and the box is written with status `nonoptimal`.

This certificate is independent of the threshold θ. It removes regions that cannot contain a maximizing quadrilateral even when the current candidate-value envelope is too wide to certify $f(Q) \le \theta$. This use is sound only because items 4 and 5 enclose exact pair-construction values, not loose upper majorants.

4.2 Flat and Small-Area Estimate

Let

$$A_Q = \mathrm{Area}(DCBA), \qquad h = \max\{a_2, b_2\}.$$

For the fence associated to the pair of opposite sides (DC, BA), a direct construction gives

$$L_{DC,BA} \le h.$$

Indeed, the relevant separating curve can be chosen with length no larger than the largest height of the two mobile vertices over the base line DC.

For an admissible counterclockwise convex quadrilateral in the normalized domain,

$$A_Q = \mathrm{Area}(DCB) + \mathrm{Area}(DBA) > \frac{b_2}{2}, \qquad A_Q = \mathrm{Area}(DCA) + \mathrm{Area}(CBA) > \frac{a_2}{2}.$$

Hence $h \le 2A_Q$, and therefore

$$\frac{L_{DC,BA}}{\sqrt{A_Q}} \le \frac{h}{\sqrt{A_Q}} \le 2\sqrt{A_Q}.$$

Consequently, if interval arithmetic proves

$$A_{\mathrm{ub}} \le \frac{\theta^2}{4},$$

then the whole box has normalized shortest fence value at most θ. This is implemented by `box\_small\_area\_certifies\_low` and recorded with status `flat\_area`.

For the threshold used below,

$$\theta = 1.0496, \qquad \theta^2 = 1.10166016, \qquad \frac{\theta^2}{4} = 0.27541504.$$

4.3 Angle Consequence

The vertex fence at an angle α has normalized upper bound $\sqrt{\alpha}$. Thus, if the smallest angle of $DCBA$ is less than θ^2, the quadrilateral is already below threshold. Any possible above-threshold optimizer must therefore have all four angles at least θ^2, so its largest angle is at most

$$2\pi - 3\theta^2.$$

For $\theta = 1.0496$, this is

$$2.978204827 \text{ radians} = 170.638567^\circ,$$

or equivalently the gap from a straight angle is at least

$$3\theta^2 - \pi = 0.163387826 \text{ radians}.$$

This angle observation explains why very flat quadrilaterals should not be optimal. The implemented small-area certificate above is the direct quantitative exclusion used in the current computation.

5 Functional Bounds

5.1 Vertex Bounds

For a vertex V with neighboring vertices P, Q, the vertex item is

$$F_V = \sqrt{\angle PVQ}.$$

The angle is computed as

$$\mathrm{atan2}\big(|(P-V)\times(Q-V)|,\ (P-V)\cdot(Q-V)\big).$$

If a coarse interval contains a degenerate vertex case where `atan2` becomes indeterminate, the value interval is clamped conservatively to $[0, \pi]$, and derivative information is treated as indeterminate. In centered mode this forces a conservative fallback to the natural enclosure.

5.2 Opposite-Pair Candidates

There are two exact opposite-pair candidate values: (DC, BA) and (CB, AD). Let the two supporting line directions make acute angle $\gamma \in [0, \pi/2]$, and let $A_Q = \text{Area}(DCBA)$. The implementation uses two algebraically identical formulas.

When $\inf \gamma >$ `GAMMA\_SPLIT`, the apex form is used. Let O be the intersection point of the two supporting lines, and let T_0, T_1 be the areas of the two triangles formed by the non-pair edges and O. Then

$$F^2_{\text{pair}} = \frac{\gamma(T_0 + T_1)}{A_Q}.$$

Near parallelism, the symmetric formula is used:

$$F^2_{\text{pair}} = \frac{\gamma}{\sin \gamma} \frac{p_a + p_b}{2A_Q},$$

where each p is a product of distances from the endpoints of one non-pair edge to the two opposite supporting lines. This form avoids the unstable intersection point O when γ is small.

5.3 Natural and Centered Enclosures

The natural enclosure evaluates each expression directly over interval boxes. The centered enclosure uses a mean-value form

$$F_i(m) + \sum_{k=0}^{3} \partial_k F_i(X)\,[-r_k, r_k],$$

where m is the coordinate midpoint and r_k is half the coordinate width. The derivatives $\partial_k F_i(X)$ are obtained by forward-mode automatic differentiation over Arb balls. In centered mode the code combines the centered and natural enclosures conservatively: if both are finite and overlap, their intersection is used; if they are finite but disjoint, their hull is used; if one is non-finite, the finite one is used.

6 Function Reference

This section lists the implemented functions by module. Static functions are included because they encode important proof logic.

6.1 `geom.c/geom.h`

`Function`	Purpose
`jet_init`	Initializes the value ball and all four derivative balls of a jet.
`jet_clear`	Clears all Arb storage owned by a jet.
`jet_set`	Copies a jet value and its derivative enclosures.
`jet_set_var`	Seeds one moduli coordinate as an automatic-differentiation variable with derivative 1 in coordinate k.
`jet_set_const`	Sets a constant Arb value and zero derivatives.
`jet_set_si`	Sets a signed integer constant and zero derivatives.
`jet_add`	Adds two jets componentwise.
`jet_sub`	Subtracts two jets componentwise.
`jet_neg`	Negates value and derivatives.

Function	Purpose
jet_mul	Multiplies jets using the product rule; temporaries allow aliasing.
jet_div	Divides jets using the quotient rule; non-finite balls are preserved when the denominator straddles zero.
jet_sqr	Squares a jet and applies $d(a^2) = 2a\,da$.
jet_sqrt	Applies square root with derivative $da/(2\sqrt{a})$. Degenerate input may produce non-finite derivative enclosures, which higher layers handle conservatively.
jet_abs	Computes $\lvert a\rvert$. If the value straddles zero, the value becomes $[0, \sup\lvert a\rvert]$ and derivatives receive sign uncertainty.
jet_atan2	Computes $\operatorname{atan2}(y, x)$ and its derivative $(x\,dy - y\,dx)/(x^2 + y^2)$.
jet_mul_si	Multiplies a jet by a signed integer.
jet_mul_arb	Multiplies a jet by a scalar Arb constant.
jpt_init	Initializes a two-coordinate jet point.
jpt_clear	Clears a jet point.
jpt_sub	Subtracts two jet points.
jet_cross	Computes the two-dimensional cross product of two jet vectors.
jet_dot	Computes the dot product of two jet vectors.
trap_cross	Scalar Arb cross product for raw vector components.
trap_dot	Scalar Arb dot product for raw vector components.

6.2 series.c/series.h

Function	Purpose
series_build	Static initializer that computes Taylor coefficients for $\gamma/\sin\gamma$ as rational numbers and stores them as high-precision Arb balls.
ensure_coeffs	Static guard that lazily initializes the coefficient table; protected by an OpenMP critical section when OpenMP is enabled.
series_cleanup	Clears cached coefficient balls. Called by the CLI and tests before exit.
ghi_ball	Static helper that turns the upper endpoint of a γ interval into a thin Arb ball.
clamp_nonneg	Static helper that intersects γ with $[0, \pi/2]$, valid because the pair angle is acute by construction.
gamma_over_sin_series	Taylor-series enclosure for $g(\gamma) = \gamma/\sin\gamma$, with a rigorous positive tail bound.
gamma_over_sin_value	Monotone endpoint enclosure for $g(\gamma)$, avoiding the removable $0/0$ singularity.
gamma_over_sin_deriv	Encloses $g'(\gamma)$ by direct formula away from zero and by series near zero.
jet_gamma_over_sin	Applies g to a jet using the value enclosure and derivative enclosure.

6.3 functionals.c/functionals.h

Function	Purpose
vbound	Static helper for the vertex bound $\sqrt{\angle PVQ}$.
pdist	Static helper for the distance from a point to a line, used in the parallel-safe pair formula.
tri_area	Static helper for triangle area $\frac{1}{2}\|(Q-P)\times(O-P)\|$.
pair_bound	Static helper implementing the exact opposite-pair candidate value. It selects the apex formula or the parallel-safe formula unless a test forces one; both formulas represent the same construction.
compute_items	Static builder for the six candidate-value jets from the four coordinate jets. In public notation it constructs $A_Q = (b_2 + b_1a_2 - a_1b_2)/2$; in the legacy implementation slots this is $(c_2 + c_1d_2 - d_1c_2)/2$.
functionals_eval	Public routine that encloses all six candidate values on a coordinate box, in natural or centered form.
functionals_pair_forced	Test hook that evaluates one pair candidate at a point while forcing the apex or parallel-safe formula.
functionals_min	Encloses the minimum envelope over the six candidate values. Non-finite item enclosures are ignored; if all six are non-finite, the result is $+\infty$, so the search cannot certify the box.
functionals_opposite_pairs_disjoint	Encloses the two exact opposite-pair candidate values and returns true only when both finite intervals are disjoint, certifying failure of the pair-equality necessary condition for optimality.

6.4 admissible.c/admissible.h

Function	Purpose
set_box	Static helper converting double endpoints into an Arb interval.
sqlen	Static helper computing squared vector length in Arb arithmetic.
box_certainly_inadmissible	Returns true only when a whole box is proved inadmissible. It checks CCW convexity through cross products, side lengths through squared distances, and optionally the reflection half-domain condition $a_1+b_1 \geq 1$ (stored internally as $c_1 + d_1 \geq 1$).
box_small_area_certifies_lo	Returns true only when the interval upper bound for area satisfies $A_{\mathrm{ub}} \leq \theta^2/4$, so the estimate $L_{DC,BA}/\sqrt{A_Q} \leq 2\sqrt{A_Q}$ proves the box below threshold.

6.5 search.c/search.h

Function	Purpose
search_root_box	Writes the full coordinate domain $[0,2] \times [0,1] \times [-1,1] \times [0,1]$.
st_init	Static stack initializer for pending boxes.
st_free	Static stack destructor.
st_push	Static stack push with geometric reallocation.
st_pop	Static stack pop; returns false when empty.
widest_coord	Static helper selecting the coordinate with largest root-span-scaled width.
classify	Static core classifier. It applies inadmissibility, optional flat-area certification, optional pair-equality nonoptimality, functional certification, optional adaptive precision, survivor flooring, and child generation.
stats_add_leaf	Static statistics accumulator for a final leaf.
run_serial	Static depth-first serial driver over a seeded stack.
run_parallel	Static OpenMP driver using a shared stack and critical sections for stack/statistics updates.
run_stack	Static dispatcher that initializes statistics and selects serial or parallel execution.
search_run	Starts a search from the root box.
search_run_seeded	Starts a search from explicit seed boxes, used by `--refine`.

6.6 threshold.c/threshold.h

Function	Purpose
parse_exp10	Static helper parsing the exponent part of a decimal literal.
threshold_parse_fmpq	Parses a finite decimal literal, including optional scientific notation, as the exact rational represented by its characters.
threshold_set_arb	Converts an exact threshold literal to an Arb ball at the requested precision.
threshold_arf_leq	Converts an Arb upper endpoint, represented as an `arf\_t`, to an exact rational and compares it with the exact threshold rational.
threshold_area_ub_leq	Compares an exact Arb area upper endpoint with the exact rational $\theta^2/4$.
threshold_to_double	Converts the literal to a double for display and adaptive-precision heuristics only; it is not used for certification.

6.7 cert.c/cert.h

Function	Purpose
status_name	Static conversion from leaf status enum to JSON status string.
cert_write_meta	Writes the schema-3 metadata line containing exact threshold literal, threshold display approximation, form, half-domain flag, auxiliary flags, precision provenance, root box, and splitter spans.

Function	Purpose
cert_write_leaf	Leaf callback that writes one JSON line containing box, status, active item, enclosure, and precision.
parse_meta_line	Static parser for the metadata JSON line.
parse_line	Static parser for certificate leaf JSON lines.
parse_status	Static parser for status strings; accepts low as an alias for certified, and recognizes flat_area and nonoptimal.
cmp_double	Static total ordering helper for doubles used in sorting boxes.
cmp_box_arrays	Static lexicographic comparison of box endpoints.
leaf_cmp	Static comparator for sorting certificate leaves.
widest_coord_cert	Static copy of the deterministic splitting rule used by the verifier.
meta_compatible	Static check that two certificate files assert the same global claim parameters before assembly.
verify_leaf_claim	Static local audit of one leaf: discarded boxes must still be certainly inadmissible; certified and flat-area boxes must recompute below θ; nonoptimal boxes must recompute disjoint opposite-pair intervals; survivors require no value claim.
verify_cover_node_slice	Static recursive tiling audit over the leaves contained in the current node. Missing children fail immediately; the verifier no longer descends empty subtrees.
cover_node_slice_structure	Static structural tiling check used by assembly, without rechecking local analytic claims.
cert_load_survivors	Loads survivor boxes from a certificate file for a refinement run.
cert_verify	Public verifier combining parsing, duplicate detection, global coverage checking, and local claim recomputation.
cert_assemble	Public assembler that replaces survivor leaves through a refinement chain and writes one full-domain JSONL certificate.

6.8 fence_validate.c

Function	Purpose
usage	Static helper printing command-line options.
main	Parses options, dispatches point evaluation, verification, certificate assembly, full search, or survivor refinement; writes certificates and prints statistics.

6.9 Python Tools and Tests

Function or file	Purpose
analyze_cert.py	Summarizes certificates safely. It reports survivor volume and separates certainly-admissible survivor cores from boundary-straddling survivors.

Function or file	Purpose
tests/test_trap.c: ok	Records a unit-test pass/fail result.
tests/test_trap.c: ub, lb	Convert Arb upper and lower endpoints to doubles for tests.
tests/test_trap.c: approx	Checks that an Arb enclosure lies within a tolerance of a reference value.
tests/test_trap.c: test_Tstar	Checks the reference trapezoid values and active pair item.
tests/test_trap.c: test_pair_forms	Checks agreement of the two opposite-pair formulas.
tests/test_trap.c: test_series	Checks $\gamma/\sin\gamma$ series enclosures against direct Arb evaluation.
tests/test_trap.c: test_inadmissible	Checks conservative inadmissibility on known examples.
tests/test_trap.c: test_interior_certify	Checks that an interior sub-threshold box certifies.
tests/test_trap.c: test_pair_equality_certificate	Checks that a box with disjoint opposite-pair intervals is certified nonoptimal, while a small box around T^* is not rejected by this certificate.
tests/test_trap.c: test_flat_area_certificate	Checks that small-area boxes are certified low and that a small box around T^* is not eliminated by the flat-area estimate.
tests/test_cert_cli.sh	Builds a tiny certificate, verifies it from metadata alone, checks that a wrong command-line theta fails with FAIL[meta], deletes one leaf, and checks that verification fails quickly with a coverage error.

7 Command-Line Interface

The executable is fence_validate. The principal options are:

--theta T Exact decimal threshold for certification. The literal is parsed as a rational for proof comparisons.

--wfloor W Scaled-width floor at which unresolved boxes become survivors.

--prec P Starting Arb precision in bits.

--maxprec M Cap for adaptive precision.

--form natural|centered Interval enclosure mode.

--half Use the optional reflection half-domain $a_1 + b_1 \geq 1$ (stored internally as $c_1 + d_1 \geq 1$).

--serial Force single-threaded execution.

--adaptive Retry marginal unresolved boxes at higher precision.

--pair-eq-cert Certify boxes as nonoptimal when the two exact opposite-pair candidate intervals are disjoint.

--flat-area-cert Certify boxes as flat_area when the small-area estimate proves normalized fence value below θ.

`--max-leaves N` Debugging cap. Output from capped runs should not be treated as a complete certificate.

`--out FILE` Write a JSONL certificate stream.

`--stats` Print summary statistics.

`--verify FILE` Re-check metadata, local claims, and full dyadic coverage. The file supplies θ, `form`, and `half`; command-line values for those fields are optional consistency checks.

`--assemble FILE... --out OUT` Assemble a base certificate and refinement files into one full-domain certificate.

`--eval b1 b2 a1 a2` Print the six candidate values and minimum at a point.

`--refine FILE` Seed the search from survivor boxes in a previous certificate.

8 Experiments

8.1 Build and Machine

The current threshold experiment used serial execution with centered enclosures, precision 70, and both auxiliary certificates:

```
make OMP=0
```

Machine characteristics for the timings:

Host	`beni-ThinkPad-P53`
OS/kernel	Ubuntu 24.04, Linux `6.11.0-29-generic`
CPU	Intel Core i7-9750H at 2.60 GHz, 6 cores / 12 threads
Memory	31 GiB RAM, 29 GiB swap
Compiler	Ubuntu GCC `13.3.0`
FLINT/Arb	FLINT `3.0.1`, merged Arb layout `<flint/arb.h>`
MPFR	`4.2.1`
GMP	`6.3.0`
Python	`3.12.9`
pdfTeX	`3.141592653-2.6-1.40.25`, TeX Live 2023/Debian
Mode	`make OMP=0`, `--serial`

8.2 Reference Trapezoid

The reference trapezoid used in diagnostics is approximately

$$A^* = (0.3582548434,\ 0.7071006812), \qquad B^* = (0.6417451566,\ 0.7071006812).$$

Point evaluation gives

$$f(T^*) \approx 1.04968581549,$$

with an opposite-pair item active.

8.3 Threshold $\theta = 1.0496$

The following sequence refines only the survivors of the previous run. Each run uses `--theta 1.0496`, `--prec 70`, `--form centered`, `--serial`, `--flat-area-cert`, and `--pair-eq-cert`.

Run	wfloor	Time (s)	Leaves	Discarded	Certified	Flat	Nonopt.	Survivors	Survivor volume
Base	0.025	79.139	215758	11555	116319	213	17884	69787	1.66385×10^{-2}
Refine 1	0.0125	117.585	360798	2967	282735	0	50065	25031	3.72991×10^{-4}
Refine 2	0.00625	35.091	119594	28	51386	0	55878	12302	1.14571×10^{-5}
Refine 3	0.003125	19.360	66134	0	17312	0	39816	9006	5.24218×10^{-7}
Refine 4	0.0015625	14.399	49864	0	10580	0	31046	8238	2.99697×10^{-8}
Refine 5	0.00078125	14.503	50504	0	9696	0	29790	11018	2.5052×10^{-9}
Refine 6	0.000390625	22.381	72276	0	12739	0	41012	18525	2.63256×10^{-10}

The commands were:

```
./fence_validate --theta 1.0496 --wfloor 0.025 --prec 70 \
  --form centered --serial --flat-area-cert --pair-eq-cert \
  --out flat_pair_cert_w0025.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.0125 --prec 70 \
  --form centered --serial --flat-area-cert --pair-eq-cert \
  --refine flat_pair_cert_w0025.jsonl \
  --out flat_pair_refine_w00125.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.00625 --prec 70 \
  --form centered --serial --flat-area-cert --pair-eq-cert \
  --refine flat_pair_refine_w00125.jsonl \
  --out flat_pair_refine_w000625.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.003125 --prec 70 \
  --form centered --serial --flat-area-cert --pair-eq-cert \
  --refine flat_pair_refine_w000625.jsonl \
  --out flat_pair_refine_w0003125.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.0015625 --prec 70 \
  --form centered --serial --flat-area-cert --pair-eq-cert \
  --refine flat_pair_refine_w0003125.jsonl \
  --out flat_pair_refine_w00015625.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.00078125 --prec 70 \
  --form centered --serial --flat-area-cert --pair-eq-cert \
  --refine flat_pair_refine_w00015625.jsonl \
  --out flat_pair_refine_w000078125.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.000390625 --prec 70 \
  --form centered --serial --flat-area-cert --pair-eq-cert \
  --refine flat_pair_refine_w000078125.jsonl \
  --out flat_pair_refine_w0000390625.jsonl --stats
```

8.4 Survivor Analysis

The postprocessor applied to the last refinement file reports that all remaining survivors are certainly admissible, with zero boundary-straddling survivor volume. The survivor union is contained in the product

$$R_A \times R_B,$$

where

$$R_A = [0.349609375,\ 0.3662109375] \times [0.6921386719,\ 0.7219238281],$$
$$R_B = [0.6337890625,\ 0.6499023438] \times [0.6921386719,\ 0.7219238281].$$

The survivor volume bound is

$$2.63256 \times 10^{-10},$$

and the farthest point of a survivor box from T^*, in the coordinate box metric used by the analysis script, is 0.023784427.

The command

```
python3 analyze_refinement_distances.py
```

computes diagonal-style Euclidean upper bounds from each refinement's survivor rectangles to the reference trapezoid. For the displayed product rectangle $R_A \times R_B$, the A- and B-bounds are farthest-corner distances from A^* and B^*, and the four-dimensional product bound is their Euclidean combination.

File	Survivors	A bound	B bound	4D product	Max leaf 4D
`flat_pair_cert_w0025.jsonl`	69787	0.898853283	0.925578382	1.2902064	0.832882346
`flat_pair_refine_w00125.jsonl`	25031	0.61938547	0.647089329	0.895747152	0.640574016
`flat_pair_refine_w000625.jsonl`	12302	0.172000006	0.172000006	0.243244742	0.159888122
`flat_pair_refine_w0003125.jsonl`	9006	0.0761032378	0.0761032378	0.107626231	0.0706509998
`flat_pair_refine_w00015625.jsonl`	8238	0.0406977567	0.0406977567	0.0575553195	0.0457130935
`flat_pair_refine_w000078125.jsonl`	11018	0.0257506677	0.0257506677	0.0364169435	0.0324385554
`flat_pair_refine_w0000390625.jsonl`	18525	0.0172802155	0.0170411685	0.024269472	0.0237844268

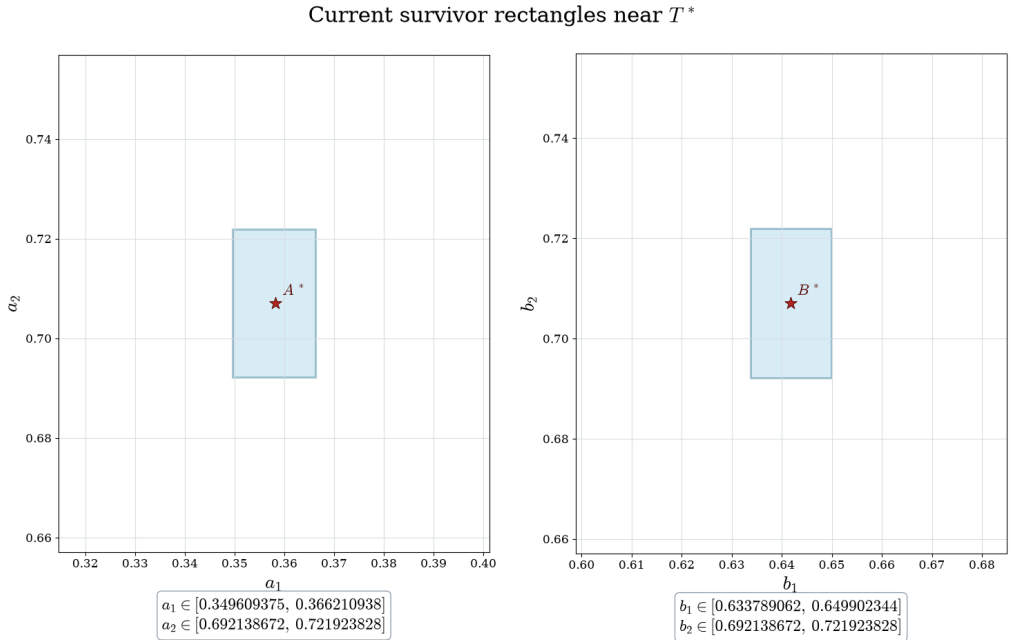

Figure 2: Equal-axis PNG export of the current survivor rectangles for A and B, together with the reference trapezoid point. The left panel is the left mobile vertex A, and the right panel is the right mobile vertex B.

8.5 Interpretation

For $\theta = 1.0496$, the current refinement sequence leaves only boxes inside $R_A \times R_B$. Outside this product rectangle, every admissible normalized quadrilateral is either certified to have shortest fence value at most 1.0496 or certified nonoptimal by the opposite-pair equality condition. Thus there is no possible above-threshold optimizer outside $R_A \times R_B$.

This is stronger than the earlier run without auxiliary certificates, but it must be read with the correct status semantics: `nonoptimal` is not a low-fence estimate. It is a separate exclusion using a necessary condition at optimality.

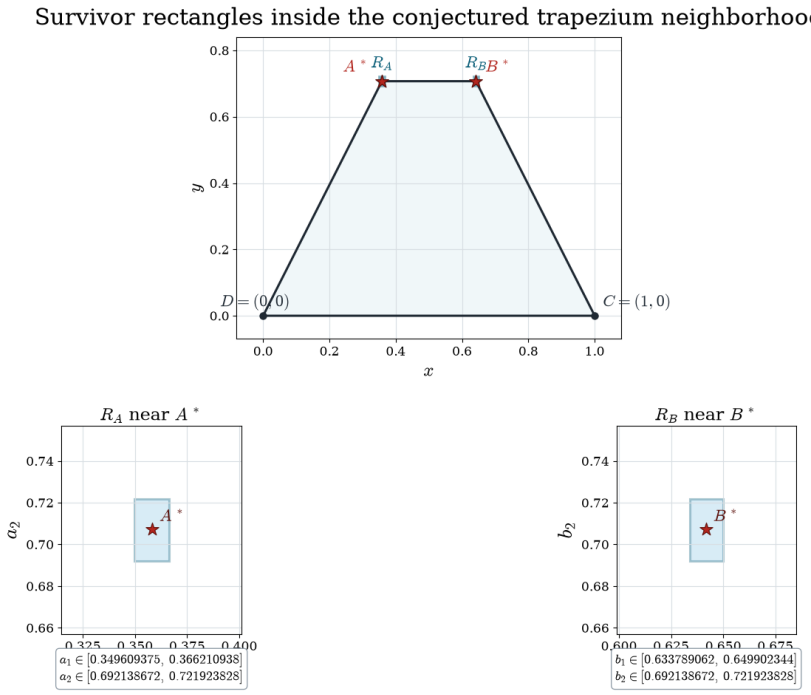

Figure 3: Overview of the conjectured optimal trapezium DCB^*A^*. The survivor rectangles R_A and R_B are drawn in place and repeated as zoomed sub-images inside the larger geometric view.

8.6 Audit Status

The listed files are incremental refinement files: each refinement replaces the survivor leaves of the preceding run. Assemble the chain before whole-domain verification:

```
./fence_validate --assemble \
  flat_pair_cert_w0025.jsonl \
  flat_pair_refine_w00125.jsonl \
  flat_pair_refine_w000625.jsonl \
  flat_pair_refine_w0003125.jsonl \
  flat_pair_refine_w00015625.jsonl \
  flat_pair_refine_w000078125.jsonl \
  flat_pair_refine_w0000390625.jsonl \
  --out flat_pair_full_w0000390625.jsonl
```

The recorded assembly produced

$$799{,}546$$

leaves. The assembled full-domain certificate was then audited by

```
./fence_validate --verify flat_pair_full_w0000390625.jsonl \
  --prec 70 --serial
```

with output

$$799{,}546 \text{ leaves}, \quad 14{,}550 \text{ discarded}, \quad 500{,}767 \text{ certified},$$
$$213 \text{ flat-area}, \quad 265{,}491 \text{ nonoptimal}, \quad 18{,}525 \text{ survivors}.$$

The dyadic partition coverage of the full root box passed, the worst rechecked certified upper bound was

$$1.0495999994684 \le 1.0496,$$

and the audit reported zero failures.

Verifying a refinement-only file as a full certificate is now a fast structural failure: the verifier reports the first missing dyadic child instead of descending an empty subtree.

The verification command may additionally include `--theta 1.0496` and `--form centered`; in the current code these are checked against the metadata rather than used to define the certificate claim.

9 Limitations and Soundness Notes

- Certification is based on upper bounds for the shortest fence. A certified box proves "below threshold", while a survivor only means that the current candidate-value enclosures were not tight enough to certify.
- Status `nonoptimal` is not an upper-bound statement. It certifies failure of the opposite-pair equality necessary condition for an optimizer. This is valid because the two pair intervals enclose exact pair-construction values. Conclusions that use `nonoptimal` boxes should be stated as exclusion of possible optimizers, not as pointwise low-fence bounds.
- Status `flat\_area` is an upper-bound statement. It uses $L_{DC,BA}/\sqrt{A_Q} \le 2\sqrt{A_Q}$ and $A_{\mathrm{ub}} \le \theta^2/4$ to prove the box below threshold.
- Coarse boxes near degeneracies can have very wide interval enclosures. This is intentional: non-finite or loose enclosures force splitting or survival, never false certification.
- The JSONL enclosure printed in a certificate is diagnostic. The verifier ignores the recorded enclosure and recomputes claims from the box, the metadata threshold, metadata form, metadata half-domain flag, and the requested verification precision.
- The metadata `theta` string is the exact proof threshold. The `theta\_approx` field and printed threshold doubles are for human readability and adaptive search heuristics only.
- A run stopped by `--max-leaves` is useful for debugging but is not a complete certificate and should fail the full coverage check.
- Parallel execution uses a shared stack, so output order may vary. The verifier sorts leaves and checks dyadic coverage, so certificate correctness does not depend on output order.